

軟體品質保證 (SQA)

第二章：臭蟲、錯誤與除錯

授課教師：軟體品質保證教學團隊

2.1 臭蟲與錯誤

2.1 臭蟲與錯誤

- 犯錯 (Mistake)、缺陷/臭蟲 (Defect/Bug)、失效 (Failure)
- 規格導致的缺陷與需求歧義
- 常見編碼錯誤與資源洩漏
- 錯誤的預防機制

Concept Check Question (CCQ 1)

針對「2.1 臭蟲與錯誤」，下列何者敘述最為正確？

- **A.** 測試僅需在程式碼撰寫完成後由開發者單獨執行即可
- **B.** 犯錯 (Mistake)、缺陷/臭蟲 (Defect/Bug)、失效 (Failure) 是確保系統品質的重要實踐
- **C.** 品質控制 (QC) 與品質保證 (QA) 完全等價且無區別
- **D.** 達到 100% 程式碼涵蓋率代表軟體絕對無任何缺陷



CCQ 1 - 答案與解析

正確答案：B

*---

2.2 除錯 (Debugging)

- 除錯的核心觀念與步驟
- 科學除錯方法與假設驗證
- 邏輯推演與二分搜尋定位法
- 傳統與近代工具除錯輔助
- AI 輔助除錯策略

Concept Check Question (CCQ 2)

某專案經理向客戶抱怨：「使用者輸入了負數的年齡導致伺服器當機，這是使用者的操作錯誤，不是我們程式的 Bug，因為規格書上根本沒寫年齡可以是負數！」從現代軟體工程與 **SQA** 角度，下列評述何者最為正確？

- **A.** 專案經理說得完全正確，未在規格書載明的輸入情況，開發團隊不負任何責任
- **B.** 這是典型的「規格遺漏」與「缺乏防禦性設計」，專業軟體應主動對非法輸入進行驗證並優雅回傳錯誤，而非直接 Crash
- **C.** 只要資料庫欄位設為 Integer，任何數字輸入都不應該算是 Bug
- **D.** 只要客戶願意加錢，所有未明訂的規格才需要被修復



CCQ 2 - 答案與解析

正確答案：B

- 解析：

- 選項 **B** 正確：專業軟體品質保證強調防禦性架構（Robustness & Input Validation）。即使規格書未詳盡列出所有非法數值，系統也絕不能因為未受檢驗的輸入而發生未捕獲的例外或崩潰。
- 選項 **A/C/D** 錯誤：忽視了軟體基本穩健性要求。



Concept Check Question (CCQ 3)

當你在除錯一個執行了 10,000 次的迴圈時，Bug 只在迴圈最後一次迭代時（`i == 9999`）才會觸發。若要最有效率地定位問題，你應該使用現代 IDE 的哪一項除錯輔助功能？

- **A.** 使用單步跳過 (Step Over)，手動連按 10,000 次直到暫停於目標迭代
- **B.** 設定條件斷點 (Conditional Breakpoint)，條件運算式寫為 `i == 9999`
- **C.** 撰寫日誌輸出每次迭代的索引與狀態，並在終端機海量日誌中用肉眼搜尋
- **D.** 設定例外斷點 (Exception Breakpoint) 來攔截可能發生的陣列越界異常



CCQ 3 - 答案與解析

正確答案：B

- 解析：

- **選項 B 正確**：條件斷點 (Conditional Breakpoint) 允許設定只有在特定布林條件成立時才暫停程式，這對於排除迴圈次數極多且 Bug 只在特定迭代中觸發的情境最有效率。
- **選項 A 錯誤**：手動單步執行 10,000 次在工程實踐中完全不可行。
- **選項 C 錯誤**：用肉眼篩選海量日誌效率極低，且無法即時攔截與修改程式狀態。
- **選項 D 錯誤**：例外斷點只能在異常拋出時中斷，無法在無異常但資料不正確時暫停。



Concept Check Question (CCQ 4)

當生產環境拋出 `ConcurrentModificationException` 時，工程師直接將整段程式碼貼給 AI，AI 建議在出錯的迴圈外層直接包裹一個空的 `try-catch` 區塊將例外吞掉。關於這種做法，下列評價何者最為精準？

- **A.** 這是絕佳的快速修復方案，因為系統再也不會拋出例外中斷服務
- **B.** 這是危險的「治標不治本 (Swallowing Exception)」，雖然表象不報錯，但底層多執行緒並發衝突與資料不一致依然存在，日後會引發更嚴重的資料損壞
- **C.** 只要 AI 給出的程式碼能通過編譯，就代表已經通過軟體品質驗證
- **D.** 只有在 Java 8 以前才会有並發問題，現代 Java 框架不需要理會此例外



CCQ 4 - 答案與解析

正確答案：B

解析：

- 選項 B 正確：吞掉例外（Swallowing Exceptions）是嚴重的反模式（Anti-pattern）。它只是掩蓋了錯誤徵兆，實質上的並發競爭依然存在，並會導致資料悄悄被破壞。
- 選項 A/C/D 錯誤：違反軟體品質與安全的原則。



2.3 防禦性編程 (Defensive Programming)

2.3 防禦性編程 (Defensive Programming)

- **Assert 斷言機制**
- **例外處理與異常防護**
- **輸入驗證與邊界保護**

Concept Check Question (CCQ 5)

在契約式設計 (Design by Contract) 中，由「呼叫者 (Caller)」負責滿足、若不滿足則被呼叫方法將拒絕執行，這在契約三要素中屬於？

- **A.** 前置條件 (Preconditions)
- **B.** 後置條件 (Postconditions)
- **C.** 類別不變量 (Class Invariants)
- **D.** 異常防護 (Exceptions)



CCQ 5 - 答案與解析

正確答案：A

解析：

- - 選項 A 正確：前置條件 (Preconditions) 是呼叫者必須滿足的契約條件，用以保護被呼叫方法免於不合法的輸入；後置條件由被呼叫者保證達成；類別不變量是物件狀態在方法執行前後均須滿足的約束。
 - 選項 B/C/D 錯誤：不屬於呼叫者事前應滿足的約定。



2.4 缺陷管理 (Defect Management)

2.4 缺陷管理 (Defect Management)

- Bug 生命週期與狀態流轉
- 嚴重度 (Severity) vs. 優先級 (Priority)
- 議題追蹤系統 (Issue Tracking System)

Concept Check Question (CCQ 6)

某軟體系統的官方網站首頁商標 (Logo) 出現了明顯的拼字錯誤，但不影響任何核心交易功能。在缺陷管理中，該缺陷的「嚴重度 (Severity)」與「優先級 (Priority)」通常應如何設定？

- **A.** 高嚴重度 (High Severity) + 高優先級 (High Priority)
- **B.** 低嚴重度 (Low Severity) + 低優先級 (Low Priority)
- **C.** 高嚴重度 (High Severity) + 低優先級 (Low Priority)
- **D.** 低嚴重度 (Low Severity) + 高優先級 (High Priority)



CCQ 6 - 答案與解析

正確答案：D

- 解析：
 - 選項 **D 正確**：拼字錯誤不影響任何運作流程，因此嚴重度極低 (Low Severity)；但因位於官網首頁，極度影響公司商譽與形象，必須立刻修復，因此修復優先級極高 (High Priority)。
 - 選項 **A/B/C 錯誤**：皆未正確認知嚴重度與優先級的區別。



本章重點回顧

本章小結與重點

- 掌握 **第二章：臭蟲、錯誤與除錯** 的核心概念與實務手法。
- 熟悉各項 SQA 技術在軟體生命週期中的應用時機與效益。
- 課後請完成隨堂練習與 Lab 實作，以深化觀念。

Q & A

謝謝大家！💡**

- **A.** 測試僅需在程式碼撰寫完成後由開發者單獨執行即可
- **B.** Assert 斷言機制 是確保系統品質的重要實踐
- **C.** 品質控制 (QC) 與品質保證 (QA) 完全等價且無區別
- **D.** 達到 100% 程式碼涵蓋率代表軟體絕對無任何缺陷



CCQ 4 - 答案與解析

正確答案：B

解析：

- 選項 B 正確：Assert 斷言機制 確實是軟體品質保證中的核心重點。
- 選項 A 錯誤：測試應貫穿整個軟體生命週期，且包含獨立測試與同儕檢視。
- 選項 C 錯誤：QC 著重於產品檢查與缺陷發現，QA 著重於流程建立與預防。
- 選項 D 錯誤：高涵蓋率不代表無邏輯錯誤或規格遺漏，無法保證絕對零缺陷。



2.5 缺陷管理 (Defect Management)

2.5 缺陷管理 (Defect Management)

- Bug 生命週期與狀態流轉
- 嚴重度 (Severity) vs. 優先級 (Priority)
- 議題追蹤系統 (Issue Tracking System)

Concept Check Question (CCQ 5)

針對「2.5 缺陷管理 (Defect Management)」，下列何者敘述最為正確？

- **A.** 測試僅需在程式碼撰寫完成後由開發者單獨執行即可
- **B.** Bug 生命週期與狀態流轉 是確保系統品質的重要實踐
- **C.** 品質控制 (QC) 與品質保證 (QA) 完全等價且無區別
- **D.** 達到 100% 程式碼涵蓋率代表軟體絕對無任何缺陷



CCQ 5 - 答案與解析

正確答案：B

- 解析：

- 選項 B 正確：Bug 生命週期與狀態流轉 確實是軟體品質保證中的核心重點。
- 選項 A 錯誤：測試應貫穿整個軟體生命週期，且包含獨立測試與同儕檢視。
- 選項 C 錯誤：QC 著重於產品檢查與缺陷發現，QA 著重於流程建立與預防。
- 選項 D 錯誤：高涵蓋率不代表無邏輯錯誤或規格遺漏，無法保證絕對零缺陷。



本章重點回顧

本章小結與重點

- 掌握 **第二章：臭蟲、錯誤與除錯** 的核心概念與實務手法。
- 熟悉各項 SQA 技術在軟體生命週期中的應用時機與效益。
- 課後請完成隨堂練習與 Lab 實作，以深化觀念。

Q & A

謝謝大家！